```
interface FavoritesControllerData {
  type: "addFavorite" | "removeFavorite"
  concertId: number
}

const getConcerts = (state: ScheduleState): Concert[] => {
  return Object.values(state.scheduleDays).flatMap(
    (scheduleDay) => scheduleDay.concerts
  )
}

const getConcert = (state: ScheduleState, concertId: number): Concert => {
  return getConcerts(state).find((concert) => concert.id === concertId)
}

export const initFavoritesChannel = (name: string): ScheduleThunk => {
  if (favoritesChannel !== undefined) {
    return
  }
  favoritesChannel = createConsumer().subscriptions.create(
    { channel: "Turbo::StreamsChannel", "signed-stream-name": name },
    {
      async received(data) {
        const parsedData = (await data.json()) as FavoritesControllerData
        scheduleStore.dispatch({
          type: parsedData.type,
          concert: getConcert(
            scheduleStore.getState(),
            parsedData.concertId
          ),
        })
      },
    }
  )
}
```

What this does is create the channel using the signed name, and then when data is received, the data tell us what type it is, and we just dispatch that action to the reducer with the correct ID.